SESI SENAI

PELO FUTURO DO TRABALHO

Objetivo

- Apresentar as estruturas básicas de JAVA

Professor

- Professor nas áreas de tecnologia, administração e educação.
- Experiência docente no SENAI, Instituto Federal Catarinense e Uniasselvi.
- Mestrando em Educação (Instituto Federal Catarinense)
  - Especialização em Educação a Distância na Educação Profissional e Tecnológica (IFC)
  - Especialização em Abordagens Ativas e suas metodologias (IFSC)
  - MBA em Gestão de Projetos (Uniasselvi)
  - MBA em Gestão de Tecnologia da Informação (IGTI)
  - MBA em Gestão Estratégica de Pessoas (Estácio)
- Graduação em Sistemas de Informação
- Licenciatura em Matemática
- Pesquisa voltada para Inteligência Artificial e inclusão.
- Atua com pesquisa, educação profissional, tecnologias educacionais e inovação na educação.

Estruturas Básicas

Identificadores: Identificadores em Java são usados para dar nomes a variáveis, rótulos, classes, pacotes, interfaces, enumerações, métodos, campos, parâmetros formais e constantes.

Programas: Um programa em Java e uma coleção de classes, dentre as quais uma pode ser a principal. A classe principal ´e conceitualmente aquela pela qual a execução se inicia.

Tipos de Dados: Os tipos de dados de Java podem ser primitivos ou referencias. Os tipos primitivos são os tipos numéricos inteiros (byte, char, short, int, long), os numéricos fracionários (float, double) e o booleano (boolean).

Estruturas Básicas

Boolean: negação (!) e as operações binarias conjunção condicional (&&), disjunção condicional (||), conjunção (&), disjunção (|), disjunção exclusiva (^), atribuição (=), comparação por igual (==) e comparação por diferente (!=).

Estruturas Básicas – Tipos numéricos

Os tipos numéricos inteiros são byte, char, short, int e long. Todos esses tipos possuem as operações de atribuição (=) e de comparação de valores, a saber: igual a (==), diferente de (!=), menor que (<), maior que (>), menor ou igual a (=<) e maior ou igual a (>=).

Os tipos numéricos int, long, float e double possuem ainda em comum as seguintes operações aritméticas: adição (+), subtração (-), multiplicação (*), divisão (/), modulo (%), pré ou pós-incremento (++), pré ou pós-decremento (--).

Estruturas Básicas – Tipos numéricos

O tipo int compreende valores inteiros do intervalo [-2.147.483.648 , 2.147.483.647]. (32 bits)
O tipo long, no fim e estão no intervalo [-9.223.372.036.854.775.808 , 9.223.372.036.854.775.807]. (64bits)
O tipo byte compreende os valores inteiros no intervalo [-128,127]. (8bits)
Os inteiros do tipo short estão no intervalo [-32.768, 32.767] (16bits)
O tipo char ´e um tipo numérico caracterizado pelos inteiros no intervalo [0,65.535], representados com 16 bits.
Valores de tipo float estão no intervalo [-3.4E38, 3.4E38] aproximadamente, com cerca de 6 dígitos significativos.
Valores de tipo double estão em [-1.7E308, 1.7E308] e tem cerca de 14 dígitos significativos, de 64 bits

Estruturas Básicas – Tipos numéricos

O tipo int compreende valores inteiros do intervalo [-2.147.483.648 , 2.147.483.647]. (32 bits)
O tipo long, no fim e estão no intervalo [-9.223.372.036.854.775.808 , 9.223.372.036.854.775.807]. (64bits)
O tipo byte compreende os valores inteiros no intervalo [-128,127]. (8bits)
Os inteiros do tipo short estão no intervalo [-32.768, 32.767] (16bits)
O tipo char ´e um tipo numérico caracterizado pelos inteiros no intervalo [0,65.535], representados com 16 bits.
Valores de tipo float estão no intervalo [-3.4E38, 3.4E38] aproximadamente, com cerca de 6 dígitos significativos.
Valores de tipo double estão em [-1.7E308, 1.7E308] e tem cerca de 14 dígitos significativos, de 64 bits

Estruturas Básicas – Constantes e Expressões

Constantes são declaradas por meio de enumerações introduzidas por enum, ou então pelo modificador final, o qual informa ao compilador que a “variável” declarada não pode ter seu valor alterado apos sua inicialização.

Expressões em Java podem ser aritméticas, logicas ou condicionais, e são formadas por operandos e operadores que podem ser unários, binários ou ternários

Estruturas Básicas

Operador	Operação
`-e`	troca sinal do operando `e` do tipo numérico
`!e`	nega logicamente o resultado de `e` do tipo `boolean`
`~e`	inteiro resultado da inversão dos bits do inteiro `e`
`++v`	valor de `v` após incrementar o seu conteúdo
`v++`	valor de `v` antes incrementar o seu conteúdo
`--v`	valor de `v` após decrementar o seu conteúdo
`v--`	valor de `v` antes de decrementar o seu conteúdo
`new t`	aloca objeto do tipo `t` e informa seu endereço
`(t)e`	converte, se possível, `e` para o tipo `t`

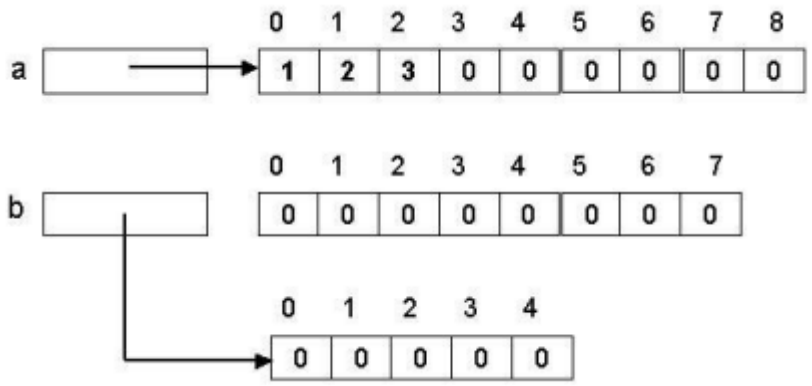
operação	Valor Produzido
`a & b`	*and* dos valores lógicos de `a` e `b`
`a | b`	*or* dos valores de `a` e `b`
`a ^ b`	*or* exclusivo dos valores de `a` e `b`
`a && b`	`false` se `a` for `false`, senão `b`
`a || b`	`true` se `a` for `true`, senão `b`
`a * b`	multiplicação dos numéricos `a` e `b`
`a / b`	divisão de `a` por `b`
`a % b`	resto da divisão do inteiro `a` pelo inteiro `b`
`a + b`	adição (numéricos) ou concatenação (strings) de `a` e `b`
`a - b`	subtração dos numéricos `a` e `b`
`a == b`	`true` se valor de `a` for igual ao valor de `b`
`a != b`	`true` se valor de `a` for diferente do valor de `b`
`a < b`	`true` se `a` for menor que `b`
`a > b`	`true` se `a` for maior que `b`
`a <= b`	`true` se `a` menor ou igual `b`
`a >= b`	`true` se `a` maior ou igual `b`
`a << c`	`a` com `c` bits deslocados para a esquerda
`a >> c`	`a` com `c` bits deslocados aritmeticamente p/ a direita
`a >>> c`	`a` com `c` bits deslocados logicamente p/ a direita
`a & b`	*and* bit a bit dos bits de `a` e `b`
`a | b`	*or* bit a bit dos bits de `a` e `b`
`a ^ b`	*or* exclusivo bit a bit dos bits de `a` e `b`

Estruturas Básicas

Prioridade	Operadores	Associatividade
1	~ ! ++ -- - (type)	direita p/ esquerda
2	* / %	esquerda p/ direita
3	+ -	esquerda p/ direita
4	<< >> >>>	esquerda p/ direita
5	< > <= >=	esquerda p/ direita
6	== !=	esquerda p/ direita
7	&	esquerda p/ direita
8	^	esquerda p/ direita
9	|	esquerda p/ direita
10	&& (curto circuito)	esquerda p/ direita
11	|| (curto circuito)	esquerda p/ direita
12	?:	direita p/ esquerda
13	= += -= *= /= %= &=	direita p/ esquerda
	^= |= <<= >>>= >>=	direita p/ esquerda

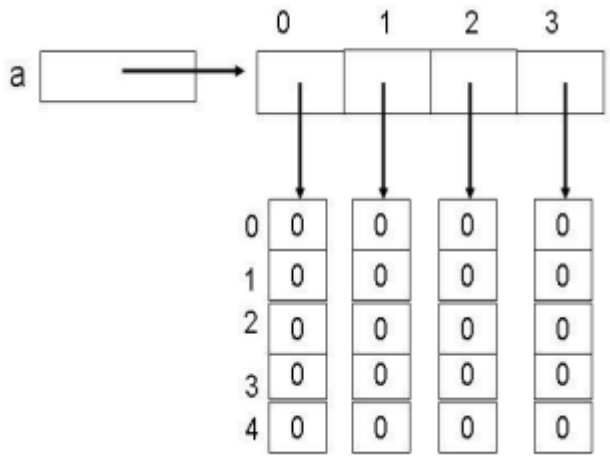
Arranjos ou Vetores

Arranjo, também chamado de vetor, é uma estrutura de dados linear unidimensional composta de zero ou mais elementos de um mesmo tipo. Arranjos são objetos criados dinamicamente, e aos quais se podem ter acesso por meio da indexação de sua referencia por um inteiro do tipo int.



Vetores Multidimensionais - Matrizes

Arranjos multidimensionais podem ser criados por meio da declaração de arranjos aninhados ou arranjo de arranjos



Comandos

Os tipos de comandos em Java são: atribuição, bloco, condicionais (if e switch), estruturas de repetição (while, do-while, for e for aprimorado), chamada de método, retorno de método (return), controle de repetição (continue e break), tratamento de exceções (try-catch-finally) e controle de acesso a regiões críticas (synchronized).

Comandos

```
1 public static void main(String[] args) {
2     int i = 10, j ;
3     if (i == 10) {
4         int j = 100;  // Erro de compilação
5         i += j;
6     }
7     j = i*10;
8 }
```

```
1 public class Switch {
2     public static void main(String[] args) {
3         int x = 0, k = 2;
4         switch (k) {
5             case 1:  x = x + 1;
6                      break;
7             case 2:  x = x + 1;
8             case 3:  x = x + 1;
9             case 4:  x = x + 1;
10                     break;
11            case 5:  x = 5;
12            default: x = x + 1000;
13        }
14        System.out.println("x = " + x);
15    }
16 }
```

```
1 public class UsoDoIf {
2     public static void main(String args[]) {
3         String s = arg[0];
4         System.out.print("Quarto caractere de arg[0] ");
5         if (s.length > 3) {
6             char c = s.charAt(3);
7             if (c == 'A')
8                 System.out.print("é um A");
9             else System.out.print("não é um A");
10        } else System.out.print("é indefinido");
11    }
12 }
```

Commander – While and doWhile

```
1  public class While {
2     public static void main(String[] arg) {
3        int[] a = {0,0,0,0,0,3,5,10,1,2,0,4};
4        int h = 0;
5        while ( h < a.length && a[h] == 0 ) h++;
6        while ( h < a.length ) {
7           System.out.println(a[h] + " ");
8           h++;
9        }
10    }
11 }
```

```
1  public class DoWhile {
2     public static void main(String[] arg) {
3        int[] a = {0,0,0,0,0,3,5,10,1,2,0,4};
4        int h = 0;
5        do {h++;} while (h < a.length && a[h-1] == 0);
6        if ( h < a.length )
7           do {
8              System.out.println(a[ht]);
9              h++;
10          } while ( h < a.length );
11    }
12 }
```

Commander – For / Rotulo

```
1  public class VarDeControle {
2    public static void main(String[] args) {
3      int i;
4      int s = 0;
5      int a [] = {1,2,3,4,5,6,7,8,9};
6      for (i = 0; i < a.length ; i++) s += a[i];
7      System.out.println("Soma de a = " + s);
8      for (int j = a.length - 1, j >= 0; j--) {
9        System.out.println(""a[" + j + "]= " + a[j]);
10     }
11   }
12 }
```

```
1  public class Continue1 {
2    public static void main(String[] args) {
3      int i = 3, j, k = 0;
4      L: while (i < 10000000) {
5           j = 2;
6           while(j < i) {
7              if (i % j ==  0) {k++; i++;continue L; }
8              j++;
9           }
10          i++;
11     }
12     System.out.print("Não-Primos = " + k);
13   }
14 }
```

Commander – Break

```
1  public class Break1 {
2     public static void main(String[] args) {
3        int i = 3, j, k;
4        L: while (i < 1000000) {
5              j = 2;
6              while(j < i) {
7                 if (i % j ==  0) { k++; break; }
8                 j++;
9              }
10             i++;
11       }
12       System.out.print("Não-Primos = " + k);
13    }
14 }
```

Comandos

1.9.12 Comando returnO comando return tem o formato return expressão e é utilizado para retornar valores em funções. Caso a função seja do tipo void, a expressão deve ser omitida.
1.9.13 Comando throwO comando throw tem o formato throw expressão e é utilizado para lançar a exceção definida pela expressão.
1.9.14 Comando tryA habilitação e o tratamento de exceções em Java são realizados por meio do comando try, que possui o formato try bloco lista de catches.
1.9.15 Comando synchronizedO comando synchronized é utilizado para sincronizar o acesso de threads concorrentes a regiões críticas e possui o formato synchronized (expressão) { comando associado }. A expressão deve ser uma referência a um objeto.

Referências

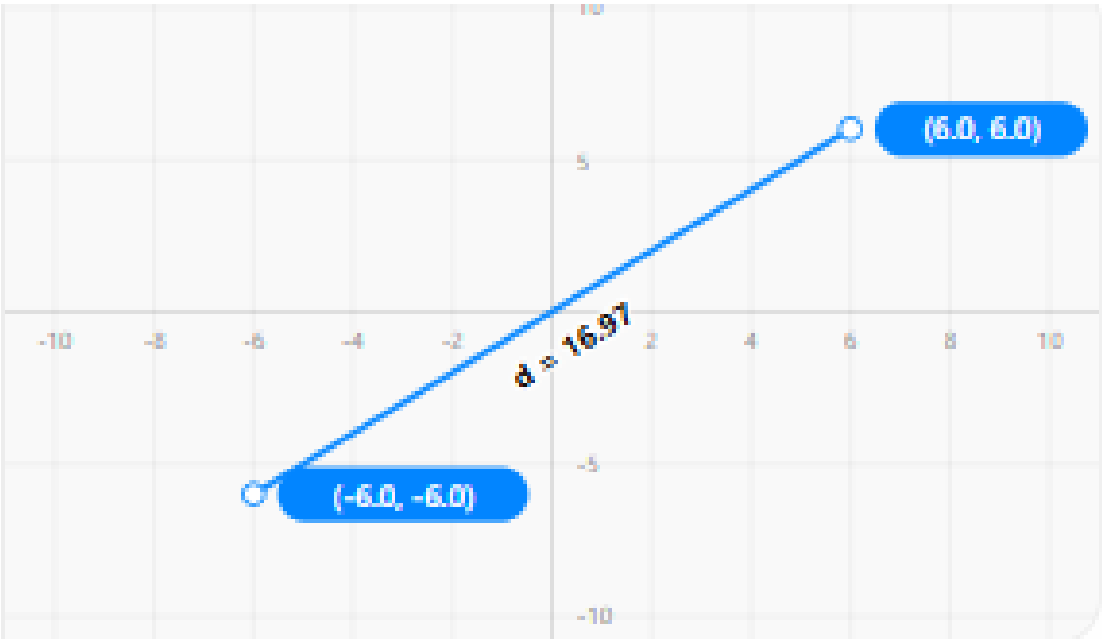
BIGONHA, Roberto da Silva. Programação modular: a linguagem Java. Belo Horizonte: [s.n.], 2021. Disponível em: Acessar PDF. Acesso em: 12 abr. 2026.

CHAGAS, Clayton Escouper das; BARUQUE, Cássia Blondet; BARUQUE, Lúcia Blondet.Java básico e orientação a objeto: volume único. Rio de Janeiro: Fundação Cecierj, 2010.

Botão Fujão Teorema de Pitágoras

O quadrado da hipotenusa é igual à soma dos quadrados dos catetos, servindo de base para o conceito de distância em espaços

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



Atividade 1 - “esperto” e configurável

- **Deixar o botão mais Distância de fuga variável**
- **Fuga proporcional:** Quanto mais perto o mouse, mais longe o botão deve pular.
- **Não sair da tela:** Garantir que o botão sempre fique dentro do painel
- **Modo fácil/difícil**: Mostrar na tela o modo atual (Label) e Tecla para trocar dificuldade

Atividade 2 - jogo completo

- O usuário precisa clicar no botão antes dele fugir.
- **Contador de pontos** Cada clique = +1 ponto
- **Tempo limite**. Ex: 30 segundos de jogo(usar Timer)
- **Velocidade progressiva** Quanto maior o tempo, mais rápido o botão foge
- **Fim de jogo**. Mostrar pontuação final

This work is licensed under a Creative Commons license

Attribution-ShareAlike 4.0

http://creativecommons.org/licenses/by-sa/4.0/

SESI SENAI

PELO FUTURO DO TRABALHO

sesisc.org.br sc.senai.br

0800 048 1212

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC